ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



BÁO CÁO BÀI TẬP LỚN

MÔN: MẠNG NƠRON VÀ HỌC SÂU

AdaZero-DCE: Điều chỉnh số vòng lặp đường cong thích ứng cho tăng cường ảnh thiếu sáng

Giảng viên: TS. Đặng Thị Thúy An

Sinh viên: Trần Minh Sang – 24100012

Ngô Quang Thiện – 24102651

Lớp học phần: Mạng nơron và học sâu-1-1-26(N02)

Mã nguồn: `https://github.com/ZesyC/AdaZero-DCE`

Hà Nội, tháng 10 năm 2026

MỤC LỤC

1 Giới thiệu bài toán **1**
1.1 Đặt vấn đề . . . 1
1.2 Đầu vào, đầu ra và phạm vi . . . 1
1.3 Các hướng tiếp cận liên quan . . . 1
1.4 Nội dung thực hiện . . . 2

2 Xây dựng và kiểm tra dữ liệu **2**
2.1 LOLv1 và phân chia tập dữ liệu . . . 2
2.2 Ý nghĩa của các mức sáng giả lập . . . 2
2.3 LOLv2-real và kiểm soát nguồn dữ liệu . . . 3
2.4 Tiền xử lý và tránh rò rỉ dữ liệu . . . 3

3 Phương pháp AdaZero-DCE **3**
3.1 Đường cong tăng cường của Zero-DCE . . . 3
3.2 Nhãn oracle trên tập train . . . 4
3.3 Đặc trưng độ sáng và head hồi quy . . . 4
3.4 Hàm mất mát và cấu hình huấn luyện . . . 5
3.5 Quy tắc suy luận . . . 5

4 Thiết kế thực nghiệm và kết quả **6**
4.1 Mục tiêu và giao thức so sánh . . . 6
4.2 Các chỉ số đánh giá . . . 6
4.3 Nguồn thực thi của báo cáo . . . 7
4.4 Kết quả trên LOLv1 eval15 . . . 7
4.5 Kết quả trên LOLv2-real Test . . . 9

5 Thảo luận **10**
5.1 Vì sao hiệu quả khác nhau giữa ảnh giả lập và ảnh thật? . . . 10
5.2 Vai trò và giới hạn của guard . . . 11
5.3 Train dài hơn có bảo đảm kết quả tăng? . . . 11
5.4 Chi phí tính toán và tính công bằng . . . 11
5.5 Hạn chế của thực nghiệm . . . 12

6 Tổ chức chương trình và khả năng sử dụng **12**
6.1 Các thành phần trong repository . . . 12
6.2 Notebook có outputs và luồng sử dụng . . . 12

7 Kết luận và hướng phát triển **13**

A Bảng kết quả chi tiết và khả năng tái lập **15**
A.1 Quy trình chạy lại . . . 15

1 Giới thiệu bài toán

1.1 Đặt vấn đề

Ảnh thu nhận trong điều kiện thiếu sáng thường có độ tương phản thấp, vùng tối khó quan sát và màu sắc không rõ ràng. Tăng cường ảnh thiếu sáng nhằm tạo ảnh dễ quan sát hơn nhưng vẫn giữ cấu trúc và nội dung của cảnh. Việc tăng độ sáng đồng loạt có thể làm các vùng vốn đủ sáng trở nên quá sáng, trong khi chi tiết tại vùng tối chưa chắc được khôi phục. Vì vậy, mức độ điều chỉnh cần phù hợp với từng ảnh đầu vào.

Zero-DCE [1] tiếp cận bài toán bằng cách ước lượng các đường cong biến đổi điểm ảnh. Mạng DCE-Net sinh bản đồ hệ số, sau đó áp dụng nhiều bước đường cong để tạo ảnh tăng cường. Mã nguồn nền trong dự án sử dụng tám bước. Cách xử lý cố định này đơn giản, nhưng đặt ra câu hỏi: liệu mọi ảnh đều cần dùng đủ tám bước, đặc biệt khi ảnh đầu vào chỉ thiếu sáng ở mức vừa phải?

Từ câu hỏi đó, nhóm xây dựng AdaZero-DCE: giữ nguyên DCE-Net và học một mạng hồi quy nhỏ để dự đoán số bước từ đặc trưng độ sáng. Mục tiêu của bài tập là triển khai một thay đổi có thể kiểm chứng, đo chênh lệch với baseline trên cùng giao thức và phân tích phạm vi mà thay đổi đó có ích. Báo cáo không đặt mục tiêu khẳng định phương pháp tốt hơn mọi mô hình tăng cường ảnh hiện có.

1.2 Đầu vào, đầu ra và phạm vi

Đầu vào là ảnh RGB $I \in [0,1]^{H\times W\times 3}$. Đầu ra gồm ảnh tăng cường $\widehat{I}$ có cùng kích thước và số vòng $n \in \{1,\dots,8\}$. Trong suy luận, hệ thống chỉ quan sát I; ảnh tham chiếu đủ sáng Y được sử dụng để đánh giá, không tham gia quyết định số vòng.

Huấn luyện được giới hạn ở head hồi quy. Trọng số DCE-Net `Epoch99.pth` luôn đóng băng. Dữ liệu có cặp chỉ được dùng để xây dựng nhãn số vòng tối ưu trên tập train. Vì head học từ nhãn được suy ra qua ảnh tham chiếu, toàn bộ phương pháp AdaZero-DCE trong báo cáo là một mở rộng có giám sát, dù backbone xuất phát từ phương pháp zero-reference. Phân biệt này giúp tránh hiểu sai tên gọi của mô hình.

1.3 Các hướng tiếp cận liên quan

Các phép biến đổi như tăng gain hoặc gamma có ưu điểm là dễ sử dụng, song mức biến đổi cần được lựa chọn phù hợp. Nhóm phương pháp dựa trên Retinex mô hình hóa quan hệ giữa phản xạ và chiếu sáng; Retinex-Net là một ví dụ học sâu gắn với dữ liệu LOL [2]. Zero-DCE học bản đồ đường cong mà không yêu cầu cặp ảnh tham chiếu trong cách huấn luyện gốc [1].

Ý tưởng điều chỉnh thích ứng đã được nghiên cứu trong các công trình khác. VAR-LIDE sử dụng thông tin visibility từ mô hình thị giác-ngôn ngữ để điều kiện hóa quá trình tăng cường đường cong trong bài toán kết hợp tăng sáng và khử mờ [4]. AdaZero-DCE trong bài tập này sử dụng đặc trưng thống kê đơn giản và một head nhỏ để chọn tiền tố của tám bước đã có. Nhóm

không tuyên bố đây là công trình đầu tiên sử dụng cơ chế thích ứng, và không đưa ra so sánh định lượng với VAR-LIDE vì chưa tái lập công trình đó trong cùng giao thức.

1.4 Nội dung thực hiện

Nhóm thực hiện đầy đủ việc tạo 1940 nhãn oracle từ 485 cặp train, huấn luyện ba head với cấu hình cố định và đánh giá fixed-8/adaptive trên cùng implementation. Hai nhóm dữ liệu kiểm tra gồm LOLv1 eval15 với ba điều kiện giả lập bổ sung, và LOLv2-real Test để kiểm tra chuyển miền. Notebook báo cáo chạy lại đánh giá bằng các checkpoint đã hoàn tất, lưu outputs, CSV và hình ảnh để truy vết số liệu [7].

2 Xây dựng và kiểm tra dữ liệu

2.1 LOLv1 và phân chia tập dữ liệu

LOLv1 là dữ liệu ảnh thiếu sáng có ảnh tham chiếu đủ sáng [2]. Dự án sử dụng nguyên phân chia `our485` gồm 485 cặp để train head và `eval15` gồm 15 cặp để đánh giá. Các cặp low/high được ghép theo tên file. Trước khi huấn luyện, pipeline kiểm tra số lượng, tập tên file và mã băm nội dung giữa hai tập để phát hiện ảnh trùng.

Với mỗi cặp train (I, Y), bốn đầu vào được tạo thành:

$$\mathcal{I}(I, Y) = \{I,\ 0.3Y,\ 0.45Y,\ 0.6Y\}. \tag{1}$$

Như vậy, số mẫu dùng tính oracle và train head là $485 \times 4 = 1940$. Ký hiệu `factor=1.0` trong artifact là ảnh low gốc I, không phải ảnh $Y \times 1.0$. Ba hệ số còn lại là ảnh GT nhân trực tiếp trên miền RGB đã chuẩn hóa.

Bảng 1: Phân chia dữ liệu được sử dụng trong dự án.

Tập	Cặp gốc	Đầu vào	Vai trò
LOLv1 our485	485	1940	Train head, tính oracle
LOLv1 eval15	15	60	Đánh giá 4 điều kiện
LOLv2-real Test	100	100	Đánh giá khái quát

2.2 Ý nghĩa của các mức sáng giả lập

Nhân ảnh GT với hệ số nhỏ hơn một tạo ra các trường hợp kiểm soát được về cường độ. Điều này giúp khảo sát việc sử dụng tám vòng có thể vượt quá mức cần thiết khi ảnh chỉ bị giảm cường độ đơn giản. Tuy nhiên, phép nhân trên ảnh sRGB không mô phỏng toàn bộ quá trình chụp ảnh thiếu sáng. Nhiễu cảm biến, sai lệch màu, mất chi tiết do phơi sáng và xử lý trong camera không được tái tạo đầy đủ.



Hình 1: Một cặp eval15 và các điều kiện giả lập trong notebook. Ba ảnh giữa được tạo từ cùng GT.

Nhận xét: Các biến thể của cùng một cặp không phải các cảnh độc lập. Tập train vẫn chỉ có 485 cảnh gốc và tập eval15 vẫn chỉ có 15 cảnh. Việc báo 1940 mẫu không đồng nghĩa tăng số cảnh lên bốn lần. Tương tự, ba seed không làm tăng số ảnh test độc lập.

2.3 LOLv2-real và kiểm soát nguồn dữ liệu

LOLv2 là phần mở rộng của LOL, được giới thiệu trong công trình về Deep Retinex Network [3]. Lần đánh giá bổ sung chỉ sử dụng 100 cặp thuộc `Real_captured/Test`. Các ảnh Low và Normal được ghép theo mã số, ví dụ `low00690.png` với `normal00690.png`. Không tải phần Train, không tạo oracle LOLv2 và không cập nhật tham số từ tập này.

Notebook tải riêng 200 ảnh test từ mirror Kaggle `tanhyml/lol-v2-dataset`, theo danh sách file đã lưu. Tổng dung lượng kiểm tra là 75,970,040 byte. Mỗi ảnh được kiểm tra kích thước byte, khả năng giải mã và lưu SHA256 để truy vết. Đây là kiểm tra tính nhất quán của dữ liệu thực tế đã dùng; chưa đối chiếu checksum của mirror với bản phát hành Google Drive của tác giả. Hạn chế nguồn dữ liệu này được giữ rõ trong báo cáo.

2.4 Tiền xử lý và tránh rò rỉ dữ liệu

Ảnh được đọc dưới dạng RGB float32 rồi chia cho 255. Không resize, crop, lượng tử hóa đầu ra trước khi tính metric hoặc điều chỉnh độ sáng đầu ra theo GT. Kích thước low và GT phải khớp. Trong lần chạy này, ảnh giữ độ phân giải gốc 600×400.

Chuẩn hóa đặc trưng dùng trung bình và độ lệch chuẩn từ 1940 mẫu train. Cùng các thống kê này được nạp lại khi đánh giá eval15 và LOLv2. Không fit lại normalization trên test. Mọi lựa chọn n được thực hiện trước khi tính PSNR/SSIM với GT. Đây là điều kiện cần để phép đo phản ánh khả năng suy luận thực tế.

3 Phương pháp AdaZero-DCE

3.1 Đường cong tăng cường của Zero-DCE

Một dạng biểu diễn đường cong thường dùng là $x + \alpha x(1 - x)$. Trong implementation đang sử dụng, quy ước dấu tương đương được viết:

$$x_t = x_{t-1} + r_t \odot (x_{t-1}^2 - x_{t-1}), \qquad x_0 = I, \tag{2}$$

trong đó r_t là bản đồ hệ số cho bước thứ t và $\odot$ là phép nhân từng phần tử. Hai cách viết tương đương khi $r_t = -\alpha_t$. Do đó, không đổi dấu hệ số chỉ để làm công thức giống cách trình bày trong tài liệu khác.

DCE-Net sinh tám bản đồ RGB từ ảnh ban đầu trong một lần forward. Với $r_t \in [-1, 1]$, phép biến đổi duy trì miền giá trị $[0, 1]$ về mặt toán học. Đạo hàm theo giá trị điểm ảnh là $1 + r_t(2x - 1)$, không âm trên miền này khi xem hệ số tại điểm ảnh là cố định. Tính chất đó lý giải vai trò của đường cong trong biến đổi cường độ, nhưng không đảm bảo chất lượng tăng cường tối ưu cho mọi cảnh.

Fixed-8 trả về x_8. AdaZero-DCE trả về x_n, tức sử dụng n bước đầu tiên của cùng chuỗi bản đồ. Phương pháp không chọn tùy ý một tập con bản đồ, không dùng lại một bản đồ cho mọi bước và không chạy lại DCE-Net tại từng bước.

3.2 Nhãn oracle trên tập train

Với mỗi đầu vào train, pipeline tính đủ tám ảnh $x_1, \ldots, x_8$ và chọn:

$$n^* = \operatorname*{argmin}_{n \in \{1,\ldots,8\}} \frac{1}{3HW} \|x_n - Y\|_2^2. \tag{3}$$

Tối thiểu hóa MSE tương đương tối đa hóa PSNR khi miền giá trị cố định. Implementation tính sai số với float64; nếu nhiều giá trị cực tiểu bằng nhau, thứ tự tìm kiếm chọn chỉ số nhỏ nhất. Oracle chỉ là lựa chọn tốt nhất trong tám ảnh ứng viên do backbone này sinh ra, không phải nghiệm tối ưu cho mọi phương pháp tăng cường ảnh.

Nhãn oracle được tính cho toàn bộ 1940 mẫu. Không áp dụng ngưỡng tối để bỏ qua việc tính nhãn. Quy tắc ngưỡng được sử dụng ở suy luận; cách tổ chức này bảo đảm cache train vẫn phản ánh đầy đủ các ứng viên. Oracle không được tính trên eval15 hoặc LOLv2 trong giao thức đánh giá hiện tại.

3.3 Đặc trưng độ sáng và head hồi quy

Tại mỗi điểm ảnh, giá trị dùng cho thống kê là $g = (R + G + B)/3$. Đây là trung bình ba kênh, không phải độ chói có trọng số theo mô hình cảm nhận. Vector đặc trưng gồm:

$$f(I) = [\mu_g, \sigma_g, h_1, \ldots, h_8] \in \mathbb{R}^{10}, \tag{4}$$

trong đó h_k là tỷ lệ điểm ảnh trong bin thứ k của histogram đều trên $[0, 1]$. Tổng tám thành phần histogram bằng một. Chuẩn hóa từng chiều theo tập train:

$$\widetilde{f}_j = \frac{f_j - \mu_j^{\text{train}}}{\sigma_j^{\text{train}} + 10^{-8}}. \tag{5}$$

Head gồm ba lớp tuyến tính theo kích thước $10 \to 128 \to 64 \to 1$, ReLU ở hai lớp ẩn và dropout 0.1 sau ReLU thứ nhất. Số tham số là:

$$(10 \cdot 128 + 128) + (128 \cdot 64 + 64) + (64 \cdot 1 + 1) = 9729. \tag{6}$$

Kích thước này nhỏ so với một mạng xử lý ảnh đầy đủ. Tuy nhiên, đặc trưng toàn cục bỏ qua bố trí không gian: hai ảnh có histogram giống nhau có thể chứa các vùng sáng tối rất khác nhau. Đây là giới hạn biểu diễn quan trọng khi diễn giải kết quả chuyển miền.

3.4 Hàm mất mát và cấu hình huấn luyện

Đầu ra liên tục $q_\theta(\widetilde{f})$ được hồi quy về nhãn n^*. SmoothL1 với tham số mặc định $\beta = 1$ có dạng:

$$\ell(e) = \begin{cases} \frac{1}{2}e^2, & |e| < 1, \\ |e| - \frac{1}{2}, & |e| \geq 1, \end{cases} \qquad e = q_\theta(\widetilde{f}) - n^*. \tag{7}$$

Hàm mất mát trung bình trên toàn bộ tập train được tối ưu bằng Adam. Cấu hình không thay đổi giữa ba seed, được trình bày ở Bảng 2.

Bảng 2: Cấu hình cố định của lần train full.

Thành phần	Giá trị
DCE-Net	Epoch99, đóng băng
Head	10–128–64–1, 9729 tham số
Activation / dropout	ReLU / 0.1
Loss	SmoothL1, $\beta = 1$
Optimizer	Adam, learning rate 5×10^{-3}
Scheduler	CosineAnnealingLR, $T_{\max} = 1200$
Epoch / batch	1200 / full batch 1940
Seed	0, 1, 2
Ngưỡng ảnh tối	mean < 0.10 thì $n = 8$

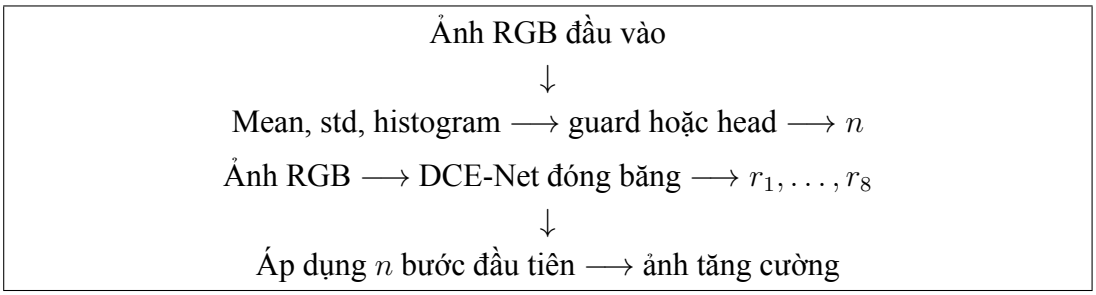
Head được train trên CPU theo implementation hiện tại; GPU T4 được dùng cho DCE-Net khi tạo oracle và đánh giá. Việc gọi toàn bộ phiên chạy là "trên Kaggle GPU" không có nghĩa mọi phép toán của head đều được đặt trên GPU. Checkpoint lưu cả state dict và thống kê chuẩn hóa để suy luận nhất quán.

3.5 Quy tắc suy luận

Với một ảnh mới, số vòng được quyết định bởi:

$$n(I) = \begin{cases} 8, & \mu_g < 0.10, \\ \mathrm{clip}(\mathrm{round}(q_\theta(\widetilde{f})), 1, 8), & \mu_g \geq 0.10. \end{cases} \tag{8}$$

Quy tắc dùng dấu nhỏ hơn nghiêm ngặt; trường hợp mean bằng 0.10 đi qua head. Dropout được tắt bằng chế độ eval. Backbone và head đều không cập nhật gradient trong đánh giá.



Nhận xét: Guard đảm bảo nhánh ảnh rất tối trả về đúng kết quả fixed-8. Điều này không chứng minh head dự đoán tốt cho các ảnh đó, bởi head đã bị bỏ qua. Khi báo cáo kết quả cần chỉ ra tỷ lệ ảnh đi qua guard để người đọc biết phần nào thực sự kiểm tra head.

4 Thiết kế thực nghiệm và kết quả

4.1 Mục tiêu và giao thức so sánh

Thực nghiệm kiểm tra ba câu hỏi: điều chỉnh số vòng có giúp các ảnh giả lập thiếu sáng vừa phải hay không; có giữ kết quả trên eval15 gốc hay không; và head LOLv1 có khái quát sang ảnh thật LOLv2 hay không. Baseline fixed-8 và adaptive dùng cùng backbone, ảnh đầu vào, GT và hàm tính metric. Khác biệt duy nhất trong phép so sánh là số bước đường cong được sử dụng.

Không đối chiếu giá trị SSIM tuyệt đối với số kỳ vọng trong kế hoạch hoặc với bảng của công trình dùng implementation khác. Đại lượng chính là chênh lệch ghép cặp giữa hai phương pháp trong cùng lần chạy. Kết quả được tính cho cả ba seed, không chọn seed tốt nhất bằng test.

4.2 Các chỉ số đánh giá

Với RGB chuẩn hóa về $[0, 1]$, PSNR được tính theo:

$$\text{MSE}(\widehat{I}, Y) = \frac{1}{3HW} \sum_{p,c} (\widehat{I}_{p,c} - Y_{p,c})^2, \qquad \text{PSNR} = -10 \log_{10}(\text{MSE}). \tag{9}$$

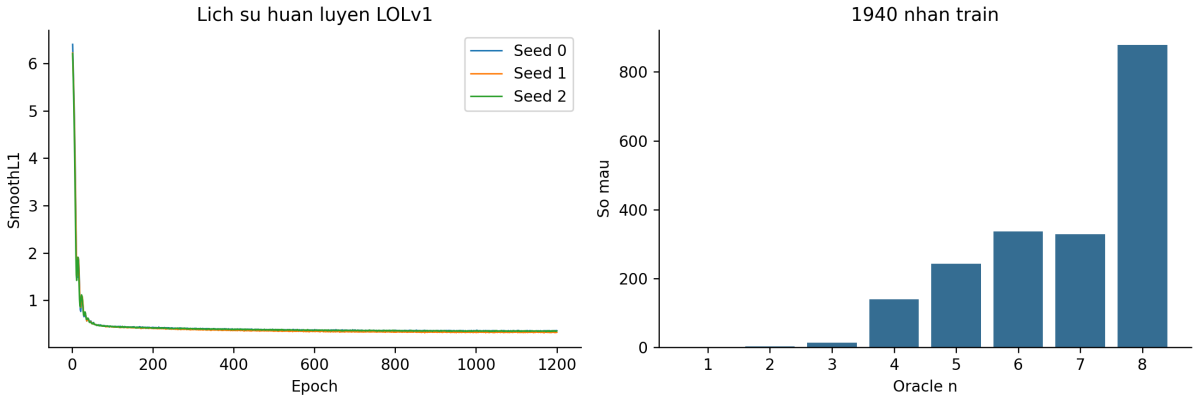
PSNR cao hơn biểu thị sai số bình phương nhỏ hơn đối với cùng GT. Metric được tính cho từng ảnh rồi lấy trung bình; không lấy MSE của toàn bộ tập để đổi sang một PSNR chung.

SSIM đánh giá quan hệ cấu trúc dựa trên thống kê cục bộ [5]. Implementation dùng `skimage.metrics.structural_similarity`, `data_range=1`, `channel_axis=2`, cửa sổ đều 7×7 và sample covariance [6]. Không dùng Gaussian window hoặc chỉ kênh Y. Do lựa chọn cửa sổ, kênh màu và cách chuẩn hóa ảnh có thể thay đổi SSIM, toàn bộ kết luận chỉ dựa trên cặp fixed-8/adaptive được đo nhất quán.

Với metric M, chênh lệch là $\Delta M = M_{\text{adaptive}} - M_{\text{fixed8}}$. Giá trị dương biểu thị cải thiện theo metric đó. Trung bình ba seed được tính từ trung bình theo ảnh của từng seed. Độ lệch chuẩn dùng mẫu số 3 (`ddof=0`); đây là biến thiên theo seed, không phải khoảng tin cậy theo ảnh.

4.3 Nguồn thực thi của báo cáo

Notebook mới được tổ chức theo các mục giải thích, cell thực thi, bảng và hình ngay sau bước tương ứng. Nó nạp nguyên ba checkpoint từ lần train full, xác minh mã băm và chạy lại inference. Lịch sử loss và oracle được đọc từ artifact train đã có, ghi rõ là dữ liệu lịch sử. Những bảng test trong báo cáo được sinh từ CSV mới của notebook này [7].



Hình 2: Loss và phân bố oracle từ lần train full đã hoàn tất; notebook hiện tại dựng lại biểu đồ từ artifact.

Nhận xét: Phân bố nhãn không đều: số mẫu oracle bằng 8 là 878, trong khi không có mẫu oracle bằng 1. Vì vậy, bài toán không cung cấp lượng thông tin như nhau cho mọi số vòng. Loss train giảm chỉ cho biết khả năng khớp các nhãn đã có; nó không đủ để kết luận chất lượng ảnh test tiếp tục tăng khi train dài hơn.

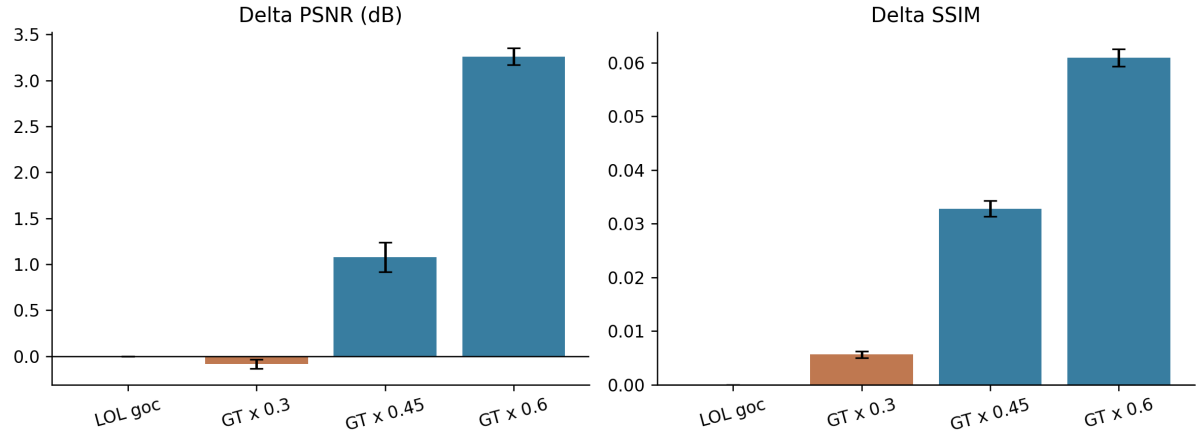
4.4 Kết quả trên LOLv1 eval15

Bảng 3: PSNR (dB) trên eval15: trung bình 3 seed, 15 ảnh/điều kiện.

Điều kiện	Fixed-8	Adaptive	Δ	Std seed
Low gốc	14.8611	14.8611	+0.0000	0.0000
GT×0.3	18.3276	18.2442	-0.0834	0.0514
GT×0.45	17.2052	18.2826	+1.0774	0.1615
GT×0.6	16.4163	19.6761	+3.2598	0.0924

Bảng 4: SSIM trên eval15: trung bình 3 seed, 15 ảnh/điều kiện.

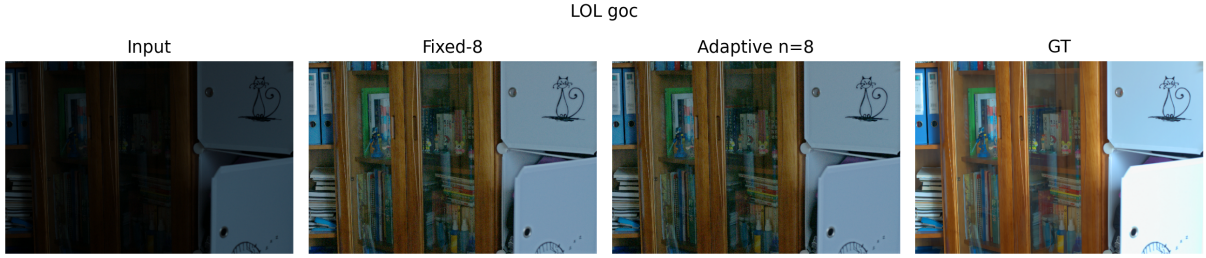
Điều kiện	Fixed-8	Adaptive	Δ	Std seed
Low gốc	0.562653	0.562653	+0.000000	0.000000
GT×0.3	0.891626	0.897264	+0.005638	0.000630
GT×0.45	0.867250	0.900064	+0.032814	0.001480
GT×0.6	0.853672	0.914620	+0.060947	0.001649



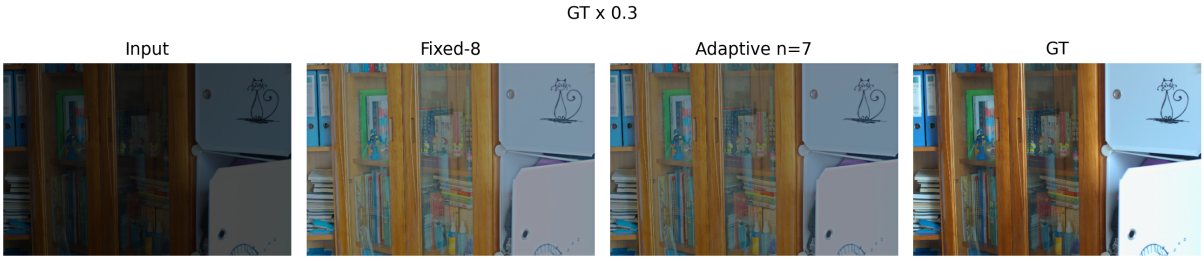
Hình 3: Chênh lệch adaptive so với fixed-8 trên eval15. Thanh sai số thể hiện độ lệch chuẩn qua ba seed.

Nhận xét: Trên ảnh low gốc, cả 15 ảnh đều có mean dưới 0.10 và được giữ ở $n = 8$. Vì hai nhánh tạo cùng đầu ra, các metric bằng nhau. Kết quả này xác nhận tác dụng bảo toàn của guard trên tập đang xét, không phải một cải thiện do head học được.

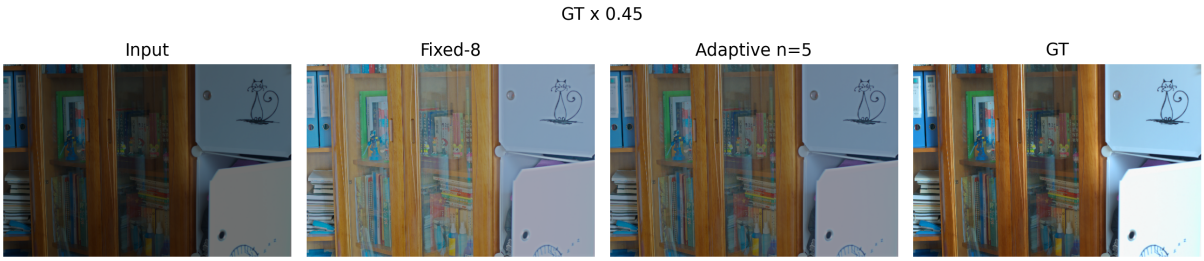
Ở mức GT×0.3, SSIM tăng nhẹ nhưng PSNR giảm. Hai metric phản ánh các khía cạnh khác nhau, nên không thể kết luận adaptive tốt hơn đồng thời theo cả hai chỉ số ở điều kiện này. Ở mức GT×0.45 và GT×0.6, PSNR và SSIM cùng tăng, với mức tăng PSNR lớn hơn ở hệ số 0.6. Đây là bằng chứng phù hợp với giả thuyết giảm số vòng có ích khi biến đổi cường độ đầu vào đơn giản và tương đối gần ảnh đủ sáng.



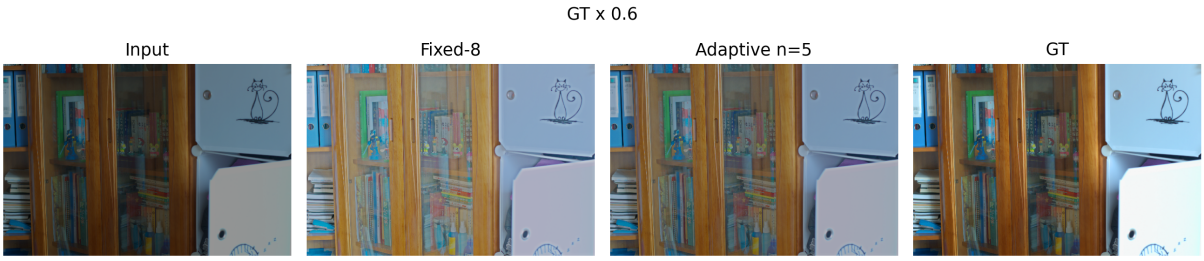
Hình 4: Ví dụ eval15 gốc: guard chọn $n = 8$, fixed-8 và adaptive trùng nhau.



Hình 5: Ví dụ GT$\times 0.3$: cùng một ảnh và một seed để minh họa, không thay cho kết quả trung bình.



Hình 6: Ví dụ GT$\times 0.45$ với đầu vào, fixed-8, adaptive và GT.



Hình 7: Ví dụ GT$\times 0.6$: đối chiếu ảnh đầu ra trong cùng điều kiện.

Các ví dụ lấy cặp đầu tiên theo thứ tự tên file và dùng seed 0, không chọn ảnh theo mức cải thiện cao nhất. Hình hiển thị được clip về miền hợp lệ để trình bày; bảng metric sử dụng trực tiếp đầu ra float theo giao thức đã nêu. Một ví dụ trực quan chỉ minh họa hành vi, không đại diện cho toàn bộ phân bố test.

4.5 Kết quả trên LOLv2-real Test

Bảng 5: PSNR (dB) trên 100 cặp LOLv2-real Test.

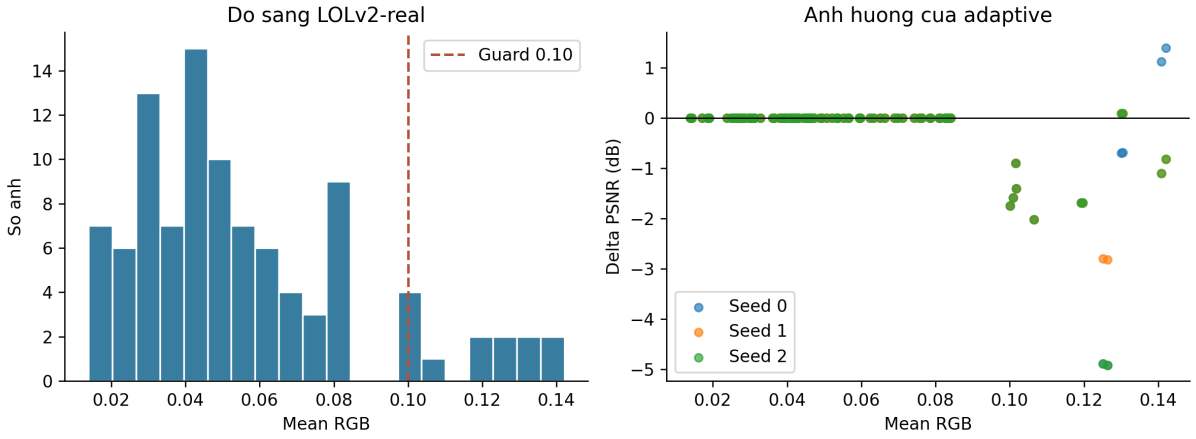
Seed	Fixed-8	Adaptive	Δ
0	18.0598	17.8628	-0.1969
1	18.0598	17.8759	-0.1839
2	18.0598	17.8340	-0.2258
Trung bình	18.0598	17.8576	-0.2022

Độ lệch chuẩn adaptive qua ba seed: 0.0175.

Bảng 6: SSIM trên 100 cặp LOLv2-real Test.

Seed	Fixed-8	Adaptive	Δ
0	0.579800	0.581476	+0.001676
1	0.579800	0.581646	+0.001846
2	0.579800	0.580941	+0.001141
Trung bình	0.579800	0.581354	+0.001555

Độ lệch chuẩn adaptive qua ba seed: 0.000300.



Hình 8: Phân bố mean RGB, ngưỡng guard và quan hệ giữa độ sáng với chênh lệch PSNR trên LOLv2-real.

Nhận xét: PSNR trung bình giảm trong cả ba seed, trong khi SSIM tăng rất nhỏ. Có 87/100 ảnh được guard giữ ở $n = 8$. Với mỗi seed, 2 ảnh tăng PSNR, 87 ảnh bằng và 11 ảnh giảm. Do phần lớn ảnh không thay đổi, trung bình toàn tập có thể che khuất mức giảm trên nhóm thực sự dùng head.

Trên 13 ảnh không đi qua guard, chênh lệch PSNR trung bình lần lượt xấp xỉ -1.515, -1.414 và -1.737 dB cho seed 0, 1 và 2. Như vậy, kết quả chưa chứng minh head khái quát tốt sang LOLv2-real. Việc SSIM tăng nhẹ không làm mất đi kết quả giảm PSNR, và không đủ để khẳng định cải thiện chất lượng cảm nhận khi chưa có đánh giá tương ứng.

5 Thảo luận

5.1 Vì sao hiệu quả khác nhau giữa ảnh giả lập và ảnh thật?

Ba điều kiện giả lập được tạo bằng nhân trực tiếp GT với một hệ số. Chúng giữ nguyên cấu trúc cảnh và nhiều đặc trưng của GT; do đó, vấn đề điều chỉnh cường độ tương đối đơn giản hơn ảnh chụp thiếu sáng thật. Trong trường hợp này, sử dụng ít bước hơn có thể giúp tránh điều chỉnh

vượt mức cần thiết. Kết quả tăng đồng thời PSNR/SSIM ở hệ số 0.45 và 0.6 phù hợp với cách giải thích đó.

Ảnh thật có thể khác về nhiễu, màu sắc, chiếu sáng không đồng đều và xử lý của camera. Vector 10 chiều không mã hóa những yếu tố này một cách trực tiếp. Hai ảnh có mean gần nhau không nhất thiết cần cùng số bước. Đây là một giải thích khả dĩ cho kết quả LOLv2, nhưng chưa phải kết luận nhân quả vì dự án chưa có ablation riêng cho từng yếu tố.

5.2 Vai trò và giới hạn của guard

Ngưỡng 0.10 giúp tránh thay đổi những ảnh rất tối theo thống kê toàn cục. Trên eval15 gốc, nó phủ toàn bộ tập; trên LOLv2, nó phủ 87%. Do đó, mức "không giảm" trên eval15 chỉ chứng minh quy tắc đã bảo toàn baseline ở những ảnh đó. Không nên sử dụng kết quả này để khẳng định head đúng cho mọi ảnh thiếu sáng.

Ngưỡng được giữ nguyên từ kế hoạch. Nhóm không chọn lại ngưỡng bằng LOLv2 Test sau khi quan sát kết quả xấu. Nếu cần nghiên cứu ngưỡng tốt hơn, phải xây dựng tập validation riêng trong train và khóa cấu hình trước lần đánh giá cuối. Việc chỉnh trực tiếp theo test sẽ làm phép đo sau đó mất tính độc lập.

5.3 Train dài hơn có bảo đảm kết quả tăng?

Không có bảo đảm như vậy. SmoothL1 tối ưu dự đoán nhãn oracle, còn kết quả cuối phụ thuộc vào bước làm tròn, guard và mức sai khác giữa các ảnh ứng viên. Một thay đổi nhỏ ở đầu ra liên tục có thể không đổi n, hoặc làm n nhảy sang số nguyên khác. Vì thế, loss giảm không dẫn đến PSNR/SSIM tăng đơn điệu.

Lần train full đã tuân thủ 1200 epoch cho 1940 mẫu và ba seed. Kết quả hiện tại cho thấy hiệu quả phụ thuộc miền dữ liệu, thay vì chỉ phụ thuộc số epoch. Chưa có thí nghiệm sweep số epoch với validation độc lập, nên báo cáo không suy diễn rằng kéo dài huấn luyện sẽ giải quyết hạn chế chuyển miền. Tương tự, ba seed ổn định về dấu chênh lệch không thay thế cho đánh giá trên nhiều bộ dữ liệu.

5.4 Chi phí tính toán và tính công bằng

Số bước đường cong có thể giảm, nhưng DCE-Net vẫn sinh đủ tám bản đồ trong một forward. Vì vậy, không thể đồng nhất giảm từ tám xuống bốn bước với tăng tốc toàn hệ thống gấp đôi. Phần tiết kiệm chỉ nằm ở các phép áp dụng đường cong; head và trích đặc trưng còn bổ sung chi phí. Chưa có benchmark latency đồng bộ GPU trên phần cứng triển khai, nên báo cáo không đưa ra tỷ lệ tăng tốc.

Notebook đánh giá dùng chung các ảnh ứng viên để tính cả fixed-8 và adaptive, thuận tiện cho so sánh chất lượng và bảo đảm cùng backbone. Thời gian notebook vì thế không đại diện cho thời gian gọi riêng hàm inference trong ứng dụng. Muốn đánh giá hiệu năng cần đo riêng đường chạy triển khai, warm-up, đồng bộ thiết bị và thống kê nhiều lần chạy.

5.5 Hạn chế của thực nghiệm

Tập eval15 nhỏ và ba ảnh giả lập của một cảnh có tương quan mạnh. Độ lệch chuẩn qua ba seed chỉ đo độ nhạy đối với khởi tạo head; chưa có bootstrap theo cảnh hoặc kiểm định thống kê. Vì vậy, các chênh lệch nhỏ cần được diễn giải thận trọng.

Nhãn oracle tối ưu PSNR, không tối ưu trực tiếp SSIM hoặc độ dễ quan sát đối với con người. Head dùng đặc trưng toàn cục và lựa chọn một số vòng chung cho toàn ảnh. DCE-Net được giữ nguyên nên khả năng sửa nhiễu, màu và chi tiết vẫn bị giới hạn bởi backbone. Ngoài ra, nguồn LOLv2 mirror chưa được đối chiếu checksum với bản tác giả. Các hạn chế này xác định phạm vi của kết luận, không được che bởi kết quả thuận lợi trên dữ liệu giả lập.

6 Tổ chức chương trình và khả năng sử dụng

6.1 Các thành phần trong repository

Bảng 7: Vai trò các thành phần hiện có và artifact bổ sung.

Thành phần	Vai trò
`zero_dce_adaptive_n.py`	Đặc trưng, head, oracle, suy luận thích ứng.
`experiment.py`	Train head và đánh giá LOLv1.
`evaluate_lolv2.py`	Kiểm tra test LOLv2 bằng checkpoint đóng băng.
`infer.py`	Tăng cường một ảnh bằng head đã lưu.
`AdaZero-DCE_code/`	Backbone và tài nguyên nền kế thừa Zero-DCE.
`notebook/`	Notebook trình bày mới, bản có outputs và CSV.
`report/`	Nguồn LaTeX, bảng và hình của báo cáo.

Trong giai đoạn hoàn thiện báo cáo, mã nguồn hiện có và checkpoint được giữ nguyên. Các tên Zero-DCE thuộc bài báo, checkpoint, mã nền hoặc tài nguyên tham chiếu vẫn giữ đúng nguồn gốc. Tên AdaZero-DCE dùng cho phương pháp mở rộng và tài liệu hiện hành; không thay tên công trình gốc để tạo cảm giác toàn bộ repository do nhóm xây dựng mới.

6.2 Notebook có outputs và luồng sử dụng

Notebook trình bày lần lượt môi trường, dữ liệu, mô hình, lịch sử train, đánh giá LOLv1, đánh giá LOLv2 và kiểm tra tái lập. Các cell gọi implementation đã có thay vì chép lại một phiên bản metric khác. Bảng kết quả xuất ra CSV để có thể kiểm tra độc lập với phần diễn giải.

Đối với một ảnh sử dụng thực tế, chương trình đọc RGB, kiểm tra miền pixel, nạp backbone cùng head và thống kê train, dự đoán n rồi áp dụng các bước đường cong. Không cần GT. Mô hình này có thể dùng làm thành phần thử nghiệm tăng sáng, nhưng kết quả LOLv2 cho thấy chưa đủ cơ sở để khuyến nghị thay fixed-8 cho mọi ảnh thật.

7 Kết luận và hướng phát triển

Bài tập đã triển khai cơ chế điều chỉnh số vòng trên backbone Zero-DCE đóng băng, huấn luyện đầy đủ 485 cặp với bốn điều kiện thành 1940 mẫu, giữ nguyên cấu hình 1200 epoch và chạy ba seed. Notebook mới tái đánh giá bằng checkpoint đã lưu và cung cấp outputs, bảng CSV cùng hình minh họa có thể truy vết.

Kết quả cho thấy lợi ích rõ ở hai điều kiện GT nhân 0.45 và 0.6. Trên eval15 gốc, hệ thống bằng fixed-8 do toàn bộ ảnh đi qua guard. Trên LOLv2-real, PSNR trung bình giảm khoảng 0.2022 dB trong khi SSIM tăng khoảng 0.001555. Kết luận phù hợp là phương pháp hữu ích trong một số điều kiện giả lập, nhưng chưa chứng minh cải thiện khái quát trên ảnh thiếu sáng thật.

Hướng tiếp theo là thiết kế validation độc lập, kiểm tra đặc trưng có thông tin không gian hoặc nhiễu, nghiên cứu mục tiêu lựa chọn số vòng phù hợp hơn và đo latency triển khai. Đây là các đề xuất nghiên cứu tiếp, chưa được thực hiện trong báo cáo. Mọi thay đổi cần được quyết định bằng train/validation và giữ tập test cho đánh giá cuối, thay vì chỉnh mô hình theo các kết quả test đã quan sát.

TÀI LIỆU THAM KHẢO

[1] C. Guo và cộng sự, "Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement," CVPR, 2020. https://github.com/Li-Chongyi/Zero-DCE.

[2] C. Wei, W. Wang, W. Yang và J. Liu, "Deep Retinex Decomposition for Low-Light Enhancement," BMVC, 2018. https://daooshee.github.io/BMVC2018website/.

[3] W. Yang, H. Huang, W. Wang, S. Wang và J. Liu, "Sparse Gradient Regularized Deep Retinex Network for Robust Low-Light Image Enhancement," IEEE Transactions on Image Processing, 2021. https://github.com/flyywh/CVPR-2020-Semi-Low-Light.

[4] W. Dong, H. Zhou, J. Lin và J. Chen, "Zero-Reference Joint Low-Light Enhancement and Deblurring via Visual Autoregressive Modeling with VLM-Derived Modulation," arXiv:2511.18591, 2025. https://arxiv.org/abs/2511.18591.

[5] Z. Wang, A. C. Bovik, H. R. Sheikh và E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," IEEE Transactions on Image Processing, 13(4), 600–612, 2004.

[6] scikit-image, tài liệu skimage.metrics.structural_similarity. https://scikit-image.org/docs/stable/api/skimage.metrics.html. Truy cập ngày 19/10/2026.

[7] Nhóm thực hiện, "AdaZero DCE Complete Evaluation Report," notebook Kaggle và các bảng CSV đi kèm, ngày 19/10/2026. https://www.kaggle.com/code/sangtranminh211/adazero-dce-complete-evaluation-report. Notebook riêng tư; bản đã thực thi được lưu trong thư mục notebook/ của dự án.

A Bảng kết quả chi tiết và khả năng tái lập

Bảng 8: Kết quả chi tiết từng seed trên eval15. Baseline xem bảng chính.

Điều kiện	Seed	PSNR	ΔPSNR	SSIM	ΔSSIM	$\bar{n}$
Low gốc	0	14.8611	+0.0000	0.562653	+0.000000	8.000
Low gốc	1	14.8611	+0.0000	0.562653	+0.000000	8.000
Low gốc	2	14.8611	+0.0000	0.562653	+0.000000	8.000
GT×0.3	0	18.3133	-0.0143	0.897251	+0.005625	7.400
GT×0.3	1	18.1901	-0.1374	0.898042	+0.006416	7.267
GT×0.3	2	18.2291	-0.0984	0.896499	+0.004873	7.333
GT×0.45	0	18.1534	+0.9482	0.899351	+0.032101	6.600
GT×0.45	1	18.5104	+1.3052	0.902125	+0.034875	6.733
GT×0.45	2	18.1841	+0.9789	0.898717	+0.031467	6.667
GT×0.6	0	19.7406	+3.3244	0.914977	+0.061305	6.133
GT×0.6	1	19.7421	+3.3259	0.916436	+0.062764	6.067
GT×0.6	2	19.5455	+3.1292	0.912446	+0.058773	6.133

A.1 Quy trình chạy lại

Cài thư viện theo `requirements.txt`. Để tái lập huấn luyện head, dùng `experiment.py` với `--train-pairs 485 --epochs 1200 --seeds 0 1 2`. Đây là lệnh huấn luyện; notebook báo cáo không gọi lệnh này. Để chỉ kiểm tra kết quả, sử dụng notebook đã cung cấp cùng hai nguồn output Kaggle của lần train full và lần đánh giá LOLv2 trước đó.

Notebook lần lượt kiểm tra SHA256, tải dữ liệu test, chạy inference, tính metric, xuất CSV và kiểm tra sai khác so với artifact trước trong sai số tuyệt đối 10^{-6}. Tất cả cell cần hoàn tất và cell cuối phải in thông báo PASS. Bản có outputs được tạo bằng thực thi `nbclient` trên Kaggle rồi tải về, không chèn số liệu thủ công.

Các bảng trong báo cáo được sinh từ CSV của lần chạy này. Manifest lưu seed, mã băm checkpoint, phiên bản PyTorch, GPU và số bản ghi. Lịch sử loss được đọc từ lần train full trước đó; không diễn giải đồ thị này là một lần huấn luyện mới. Thời gian inference trong notebook bao gồm xử lý và tính metric, không phải phép đo latency triển khai.